

API System Documentation: Account and Message Services (Spring Boot Refactor)

Jaehoon Song

May 18, 2026

1 Introduction

This document provides the formal specifications for the Account and Message management system, acting as the backend for a social media application. It details the HTTP endpoints, expected request formats, business validation logic, database integration patterns, and standardized response structures required for the implementation of the backend services.

1.1 Architecture

The system is built utilizing a robust **3-Layer Architecture**, modernized to leverage the **Spring Boot Framework**. It utilizes Spring to inject dependencies and autowire functionality using Spring annotations:

- **Controller Layer:** Manages all incoming HTTP requests via Spring Web MVC (e.g., `@RestController`). It handles routing, automatically parses JSON payloads, and returns formatted JSON responses with corresponding HTTP status codes.
- **Service Layer:** Encapsulates all business logic and strict validation rules. Annotated with `@Service`, it acts as the intermediary between the controllers and repositories, ensuring data integrity.
- **Repository Layer:** Abstracts all direct database interactions using **Spring Data JPA** and `@Repository` interfaces. It replaces raw JDBC and safely executes CRUD operations against the in-memory H2 database.

1.2 Database Entities

Figure 1 shows a compact **Crow's Foot** (IE) style ERD. The relationships read as follows: “each **Message** has exactly one **Account** (postedBy)” and “each **Account** has zero or more **Messages**”.

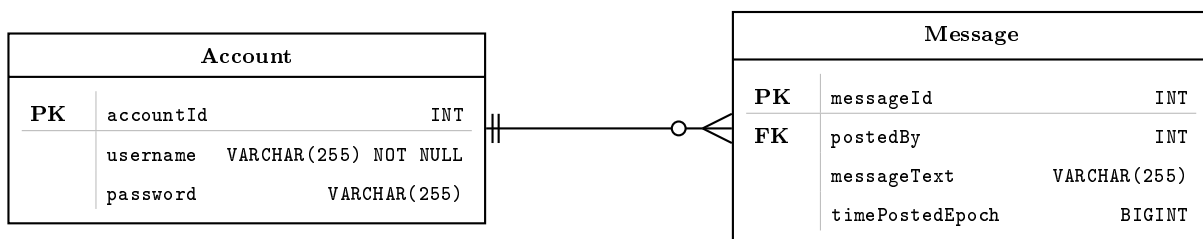


Figure 1: Crow's Foot ERD

Physical mapping (H2). accountId and messageId are INT primary keys with AUTO_INCREMENT; postedBy is an INT foreign key to Account(accountId); username is VARCHAR(255) NOT NULL UNIQUE.

1.3 Exception Handling

The system implements a centralized error handling mechanism using a Controller Advice pattern to ensure consistent HTTP responses. Custom exceptions are mapped to standard HTTP status codes:

- **ClientValidationException:** Mapped to 400 Bad Request. Thrown when request payloads fail business validation rules (e.g., blank fields, invalid lengths, or referencing non-existent entities).
- **DuplicateResourceException:** Mapped to 409 Conflict. Thrown when attempting to create a resource that violates a unique constraint (e.g., duplicate username).
- **UnauthorizedException:** Mapped to 401 Unauthorized. Thrown when authentication credentials are invalid.

2 Account Management

2.1 User Registration

A new account is created through a POST request to `localhost:8080/register`.

- **Request Body:** A JSON representation of an Account is provided, excluding the `accountId`.
- **Success Criteria:** Registration is completed if the username is not blank, the password contains at least four characters, and the username does not already exist in the database.
- **Successful Response:** A status of 200 OK is returned. The response body contains the JSON representation of the Account, including the auto-generated `accountId`. The new account is persisted to the database.
- **Error Handling:** If registration is not successful due to a duplicate username, a 409 Conflict status is returned. If the registration fails for any other validation reason, a 400 Bad Request (Client Error) status is returned.

2.2 User Login

Authentication is verified via a POST request to `localhost:8080/login`.

- **Request Body:** A JSON representation of an Account is provided, excluding the `accountId`.
- **Success Criteria:** Login is successful if the provided username and password match an exact, existing record in the database.
- **Successful Response:** A status of 200 OK is returned. The response body contains the JSON representation of the authenticated account, including the `accountId`.
- **Error Handling:** If the login is not successful (credentials do not match any existing account), a 401 Unauthorized status is returned.

3 Message Management

3.1 Create New Message

A new post is submitted via a POST request to `localhost:8080/messages`.

- **Request Body:** A JSON representation of a message is provided, excluding the `messageId`.
- **Success Criteria:** The message is persisted if the `messageText` is not blank, is not over 255 characters, and the `postedBy` field refers to a real, existing user.
- **Successful Response:** A status of 200 OK is returned. The response body contains the complete JSON of the message, including the newly generated `messageId`.
- **Error Handling:** If the creation of the message is not successful for any reason (e.g., validation failure), a 400 Bad Request (Client Error) status is returned.

3.2 Retrieve All Messages

All messages are retrieved via a GET request to `localhost:8080/messages`.

- **Functionality:** Retrieves all posts currently stored in the database.
- **Successful Response:** A status of 200 OK is returned. The response body contains a JSON list of all messages retrieved from the database. It is expected for the list to simply be empty (`[]`) if there are no messages.

3.3 Retrieve Message by ID

A specific message is retrieved via a GET request to `localhost:8080/messages/{messageId}`.

- **Functionality:** Retrieves a single post matching the exact ID path parameter.
- **Successful Response:** A status of 200 OK is returned. The response body contains the JSON representation of the identified message. It is expected for the response body to simply be empty if there is no such message.

3.4 Delete Message by ID

A message is removed via a DELETE request to `localhost:8080/messages/{messageId}`.

- **Functionality:** The identified message is permanently removed from the database.
- **Successful Response:** A status of 200 OK is returned. If the message existed prior to deletion, the response body should contain the number of rows updated (1). If the message did not exist, the response status should still be 200 OK, but the response body should be empty, as the DELETE verb is intended to be idempotent.

3.5 Update Message by ID

A message text is updated via a PATCH request to `localhost:8080/messages/{messageId}`.

- **Request Body:** A JSON payload containing a new `messageText` value to replace the message identified by the ID. The request body cannot be guaranteed to contain any other information.
- **Success Criteria:** The update is executed if the `messageId` already exists in the database, the new text is not blank, and the new text length is not over 255 characters.
- **Successful Response:** A status of 200 OK is returned. The message stored in the database should have the updated `messageText`, and the response body should contain the number of rows updated (1).
- **Error Handling:** If the update of the message is not successful for any reason, a 400 Bad Request (Client Error) status is returned.

3.6 Retrieve Messages by Account ID

All messages posted by a specific user are retrieved via a GET request to `localhost:8080/accounts/{accountId}/messages`.

- **Functionality:** Filters global messages, returning only those authored by a particular user.
- **Successful Response:** A status of 200 OK is returned. The response body contains a JSON list of all messages retrieved from the database posted by the user. It is expected for the list to simply be empty (`[]`) if there are no messages.